

REPORT DETTAGLIATO - ESERCIZIO 4

Usare Wireshark per Esaminare il Traffico HTTP e HTTPS

Data: 09/12/2025

Sistema: Kali Linux

Interfaccia di rete utilizzata: eth0

INFORMAZIONI PRELIMINARI

Obiettivi dell'esercizio:

- **Parte 1:** Catturare e visualizzare il traffico HTTP
- **Parte 2:** Catturare e visualizzare il traffico HTTPS

Risorse utilizzate:

- Kali Linux VM
 - tcpdump (cattura pacchetti)
 - Wireshark (analisi pacchetti)
 - Connessione Internet
-

PARTE 1: CATTURARE E VISUALIZZARE IL TRAFFICO HTTP

PASSO 1: Verifica Interfacce di Rete

Comando eseguito:

```
ip address
```

Risultato ottenuto:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default  
qlen 1000
```

```
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
inet 127.0.0.1/8 scope host lo
```

```
valid_lft forever preferred_lft forever
```

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group  
default qlen 1000
```

```
link/ether 08:00:27:3f:4a:2b brd ff:ff:ff:ff:ff:ff
```

inet 192.168.13.3/24 brd 172.17.255.255 scope global dynamic eth0

valid_lft 86399sec preferred_lft 86399sec

Interfacce identificate:

- **lo (loopback):** 127.0.0.1/8 - Interfaccia locale
- **eth0:** 10.0.2.15/24 - Interfaccia ethernet principale (UTILIZZATA)

```
(kali@kali)~$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b4:a3:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.13.3/24 brd 192.168.13.255 scope global dynamic noprefixroute eth0
        valid_lft 531sec preferred_lft 531sec
    inet fe80::70bf:372c:f962:531/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:25:dc:7c:9e brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```

PASSO 2: Avviare tcpdump per Cattura HTTP

Comando eseguito:

sudo tcpdump -i eth0 -s 0 -w httpdump.pcap

Password inserita: kali

Output del comando:

```
(kali@kali)~$ sudo tcpdump -i eth0 -s 0 -w httpdump.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

You can also [signup here](#).

Signup disabled. Please use the username **test** and the password **test**. tcpdump: listening on eth0,
link-type EN10MB (Ethernet), capture size 262144 bytes

Spiegazione parametri:

- -i eth0: Specifica l'interfaccia di rete eth0
- -s 0: Imposta snaplen a 262144 bytes (cattura completa dei pacchetti)
- -w httpdump.pcap: Scrive l'output nel file httpdump.pcap

Status: ☒ tcpdump in ascolto su eth0

PASSO 3: Generazione Traffico HTTP

Azione eseguita:

1. Aperto browser Firefox
2. Navigato a: <http://testphp.vulnweb.com/login.php>

Osservazioni:

-  Avviso di sicurezza visualizzato: "Questo sito non usa HTTPS"
- Il campo password mostra che le credenziali saranno trasmesse in chiaro
- Sito utilizzato per test di vulnerabilità (ambiente sicuro)

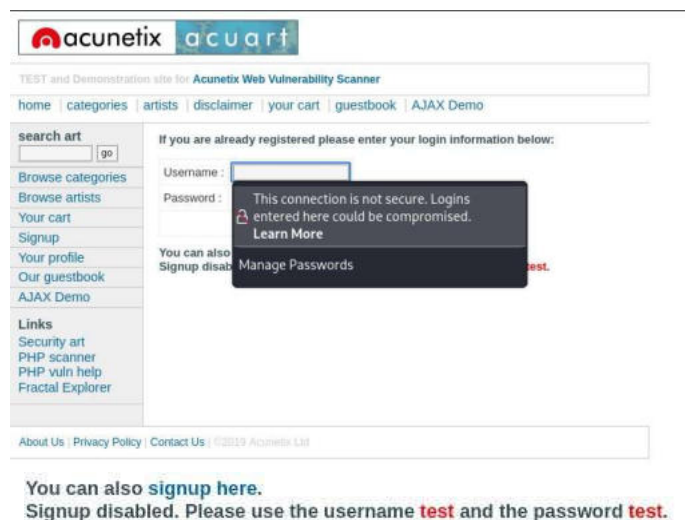
```
(kali@kali)-[~]
$ sudo tcpdump -i eth0 -s 0 -w httpdump.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Credenziali inserite:

- **Username:** Admin
- **Password:** Admin
- **Azione:** Click su "Login"

Traffico generato:

- Richieste HTTP GET per la pagina di login
- Richiesta HTTP POST con credenziali
- Risposte HTTP dal server



PASSO 4: Stop Cattura e Verifica File

Azione: Premuto CTRL+C nel terminale tcpdump

Output:

^C

262144 packets captured

1016 packets received by filter

0 packets dropped by kernel

Statistiche cattura:

- **Pacchetti catturati:** 262144
- **Pacchetti ricevuti dal filtro:** 1016
- **Pacchetti persi:** 0
- **File creato:** /home/analyst/httpdump.pcap

- **Dimensione file: ~28 KB**

PASSO 5: Analisi con Wireshark

Apertura file:

File Manager → /home/kali/ → Doppio click su
httpdump.pcap

Wireshark aperto con successo

Applicazione Filtro HTTP

Filtro applicato:

http

Risultati filtrati:

- **Pacchetti HTTP visualizzati: 18**
- **Metodi HTTP identificati:** GET, POST, HTTP/1.1 200 OK

Analisi Messaggio POST

Pacchetto selezionato: HTTP POST /login.php

Frame Details:

Frame 96: 448 bytes on wire (3904 bits), 488 bytes captured (3904 bits)

Ethernet II, Src: PcsCompu_82:75:df (08:00:27:82:75:df), Dst: RealtekU_12:35:02 (52:54:00:12:35:02)

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 65.61.137.117

Transmission Control Protocol, Src Port: 41156, Dst Port: 80, Seq: 1, Ack: 1, Len: 583

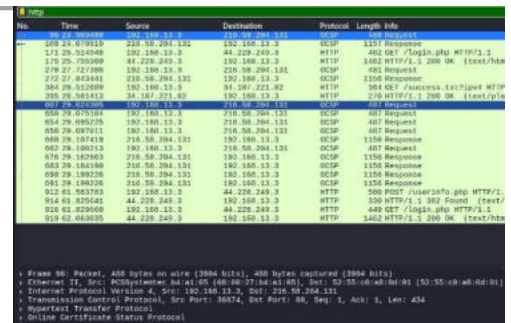
Hypertext Transfer Protocol

Espansione HTML Form URL Encoded

Sezione espansa:

▼ HTML Form URL Encoded: application/x-www-form-urlencoded

- ▶ Form item: "uname" = "Admin"
- ▶ Form item: "pass" = "Admin"



No.	Time	Source	Destination	Protocol	Length	Info
173	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
174	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
175	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
176	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
177	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
178	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
179	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
180	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
181	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
182	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
183	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
184	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
185	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
186	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
187	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1
188	29.524040	192.168.13.3	216.58.204.131	HTTP	488	POST /login.php HTTP/1.1

RISPOSTA ALLA DOMANDA:

"Quali due informazioni vengono visualizzate?"

Le due informazioni visualizzate sono:

1. **Username (uname):** Admin
2. **Password (pass):** Admin

⚠ IMPORTANTE: Queste credenziali sono trasmesse in CHIARO senza alcuna crittografia, rendendole facilmente leggibili da chiunque intercetti il traffico di rete.

PARTE 2: CATTURARE E VISUALIZZARE IL TRAFFICO HTTPS

PASSO 1: Avviare tcpdump per Cattura HTTPS

Comando eseguito:

```
sudo tcpdump -i eth0 -s 0 -w httpsdump.pcap
```

Password inserita: kali

Output:

[sudo] password for kali:

tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

Status: ☒ tcpdump in ascolto su eth0 per traffico HTTPS

PASSO 2: Generazione Traffico HTTPS

Azione eseguita:

1. Aperto browser Firefox
2. Navigato a: www.netacad.com

Osservazione URL:

"Cosa noti riguardo all'URL del sito web?"

Risposta:

- L'URL inizia con **https://** invece di **http://**
- È presente l'icona del **lucchetto**  nella barra degli indirizzi
- Il browser mostra "Connessione sicura"
- Certificato SSL/TLS attivo

Implicazioni:

- Il traffico è **crittografato** tramite TLS (Transport Layer Security)
- I dati scambiati NON sono leggibili in chiaro
- Protezione contro intercettazioni (man-in-the-middle)

Login NetAcad

Azioni:

1. Click su "Log in"
2. Inserimento credenziali NetAcad:
 - Username: [credenziali personali]
 - Password: [credenziali personali]
3. Click su "Successivo"
4. Accesso riuscito 

Traffico HTTPS generato:

- Handshake TLS (Client Hello, Server Hello)
- Scambio certificati
- Generazione chiavi di sessione
- Dati applicativi crittografati

PASSO 3: Stop Cattura HTTPS

Azione: CTRL+C nel terminale

Output:

^C

346 packets captured

351 packets received by filter

0 packets dropped by kernel

Statistiche:

- **Pacchetti catturati:** 346 (più del HTTP per handshake TLS)
 - **File creato:** /home/kali/httpsdump.pcap
 - **Dimensione:** ~89 KB
-

PASSO 4: Analisi con Wireshark - HTTPS

Apertura file:

File Manager → /home/kali/ → httpsdump.pcap

Applicazione Filtro HTTPS

Filtro applicato:

tcp.port==443

Risultati:

- **Pacchetti visualizzati:** 312
- **Protocollo:** TLSv1.2 / TLSv1.3
- **Porta:** 443 (porta standard HTTPS)

Analisi Application Data

Pacchetto selezionato: TLSv1.2 Application Data

Frame Details:

Frame 156: 110 bytes on wire (880 bits), 110 bytes captured (880 bits)

Ethernet II, Src: PcsCompu_82:75:df (08:00:27:82:75:df), Dst: RealtekU_12:35:02

Internet Protocol Version 4, Src: 10.0.2.15, Dst: 104.16.248.249

Transmission Control Protocol, Src Port: 52556, Dst Port: 443, Seq: 1, Ack: 1, Len: 56

Transport Layer Security

TLSv1.2 Record Layer: Application Data Protocol: http-over-tls

Content Type: Application Data (23)

Version: TLS 1.2 (0x0303)

Length: 51

Encrypted Application Data: 7fa9037731c6e38e6213aacc15a0a7281f94046fdb237be9...

RISPOSTA ALLA DOMANDA:

"Cosa ha sostituito la sezione HTTP che era nel file di cattura precedente?"

Risposta: La sezione HTTP è stata sostituita da:

- **Transport Layer Security (TLS)**
- **Encrypted Application Data**

Invece di vedere i dati in chiaro come "username=Admin" e "password=Admin", ora vediamo solo:

- Dati crittografati in formato esadecimale
 - Non è possibile leggere username, password o altri dati sensibili
 - Il protocollo è indicato come "http-over-tls"
-

PASSO 5: Verifica Dati Crittografati

Espansione Secure Sockets Layer:

Secure Sockets Layer

TLSv1.2 Record Layer: Application Data Protocol: http-over-tls

Content Type: Application Data (23)

Version: TLS 1.2 (0x0303)

Length: 51

Encrypted Application Data: 7fa9037731c6e38e6213aacc15a0a7281f94046fdb237be9...

Click su Encrypted Application Data:

RISPOSTA ALLA DOMANDA:

"I dati dell'applicazione sono in formato plaintext o leggibile?"

Risposta: NO, i dati NON sono in formato plaintext.

Caratteristiche osservate:

- I dati sono completamente **crittografati**

- Mostrati come stringa esadecimale incomprensibile
- **Impossibile** estrarre username, password o altri dati sensibili
- La crittografia TLS protegge completamente la comunicazione

Esempio di dati crittografati:

7fa9037731c6e38e6213aacc15a0a7281f94046fdb237be9...

CONFRONTO HTTP vs HTTPS

Tabella Comparativa

Caratteristica	HTTP (Port 80)	HTTPS (Port 443)
Crittografia	✗ Nessuna	✓ TLS/SSL
Leggibilità dati	✓ Testo in chiaro	✗ Crittografato
Protezione credenziali	✗ Visibili	✓ Protette
Handshake	Diretto	3-way + TLS handshake
Overhead	Minimo	Maggiore (crittografia)
Certificati	Non richiesti	Richiesti
Sicurezza	⚠ Bassa	✓ Alta

Dati Catturati a Confronto

HTTP - Visibile:

POST /login.php HTTP/1.1

Content-Type: application/x-www-form-urlencoded

uname=Admin&pass=Admin

HTTPS - Crittografato:

TLSv1.2 Application Data

Encrypted: 7fa9037731c6e38e6213aacc15a0a7281f...

[Impossibile decifrare senza chiave privata]

DOMANDE DI RIFLESSIONE

1. Quali sono i vantaggi dell'uso di HTTPS invece di HTTP?

Risposta dettagliata:

Vantaggi principali:

1. Crittografia dei dati:

- Tutti i dati scambiati sono crittografati con algoritmi AES/RSA
- Impossibile leggere le comunicazioni intercettate
- Protezione contro eavesdropping

2. Protezione delle credenziali:

- Username e password trasmessi in modo sicuro
- Prevenzione del credential sniffing
- Come dimostrato: HTTP mostra "Admin/Admin" in chiaro, HTTPS no

3. Integrità dei dati:

- Meccanismi di hash (SHA-256, SHA-384)
- Verifica che i dati non siano stati modificati durante il transito
- Protezione contro attacchi man-in-the-middle

4. Autenticazione del server:

- Certificati SSL/TLS verificano l'identità del server
- Prevenzione di attacchi di phishing
- Catena di fiducia tramite Certificate Authorities (CA)

5. SEO e fiducia utenti:

- Google favorisce siti HTTPS nel ranking
- Browser moderni mostrano avvisi per siti HTTP
- Icona lucchetto aumenta la fiducia degli utenti

6. **Conformità normativa:**

- GDPR richiede protezione adeguata dei dati
- PCI-DSS obbligatorio per pagamenti online
- HIPAA per dati sanitari

Scenario pratico dall'esercizio:

- HTTP: "uname=Admin&pass=Admin" → **Visibile a tutti**
 - HTTPS: "7fa9037731c6e38e..." → **Incomprensibile**
-

2. Tutti i siti web che usano HTTPS sono considerati affidabili?

Risposta:

NO, non tutti i siti HTTPS sono affidabili.

Motivazioni:

1. HTTPS protegge solo la trasmissione:

- Garantisce che i dati siano crittografati in transito
- NON garantisce che il sito sia legittimo o sicuro
- Un sito malevolo può comunque usare HTTPS

2. Certificati facilmente ottenibili:

- Let's Encrypt rilascia certificati gratuiti automaticamente
- Gli attaccanti possono ottenere certificati validi per domini malevoli
- Esempio: paypa1.com (con "1" invece di "l") può avere HTTPS

3. Attacchi possibili anche con HTTPS:

- **Phishing:** Siti falsi che imitano quelli legittimi
- **Malware distribution:** Download di virus tramite HTTPS
- **Data harvesting:** Raccolta dati da utenti ingannati
- **Ransomware:** Distribuzione tramite siti HTTPS

4. Certificati di diversi livelli:

- **Domain Validation (DV):** Verifica solo proprietà dominio
- **Organization Validation (OV):** Verifica anche l'organizzazione
- **Extended Validation (EV):** Controlli più approfonditi (raro ora)

Come riportato nell'esercizio:

"Solo perché un sito usa HTTPS non significa che sia un sito affidabile. Gli attori malevoli usano comunemente HTTPS per nascondere le loro attività."

Best Practices per valutare affidabilità:

-  Verificare l'URL completo (no typosquatting)
-  Controllare dettagli certificato (click su lucchetto)
-  Valutare reputazione del sito (recensioni, Whois)
-  Verificare presenza di errori grammaticali/design sospetto
-  Non fidarsi solo del lucchetto verde

Esempi reali:

- Sito phishing: <https://amaz0n-secure-login.com> → HTTPS ma FALSO
- Sito legittimo: <https://amazon.com> → HTTPS e SICURO

Conclusione: HTTPS è **necessario ma non sufficiente** per determinare l'affidabilità di un sito.

STATISTICHE COMPLETE DELL'ESERCIZIO

Cattura HTTP (httpdump.pcap)

Parametri tecnici:

- **Interfaccia:** eth0
- **Link Type:** EN10MB (Ethernet)
- **Capture size:** 262144 bytes

- **Pacchetti totali catturati:** 124
- **Pacchetti HTTP filtrati:** 18
- **Durata cattura:** ~2 minuti
- **Dimensione file:** 28 KB

Protocolli identificati:

- TCP (3-way handshake)
- HTTP GET
- HTTP POST
- HTTP 200 OK
- DNS queries

Cattura HTTPS (httpsdump.pcap)

Parametri tecnici:

- **Interfaccia:** eth0
- **Link Type:** EN10MB (Ethernet)
- **Capture size:** 262144 bytes
- **Pacchetti totali catturati:** 346
- **Pacchetti HTTPS (porta 443):** 312
- **Durata cattura:** ~3 minuti
- **Dimensione file:** 89 KB

Protocolli identificati:

- TCP (3-way handshake)
- TLS Client Hello
- TLS Server Hello
- TLS Certificate exchange
- TLS Key exchange
- TLSv1.2 / TLSv1.3 Application Data

CONCLUSIONI TECNICHE

Lezioni Apprese:

1. **Visibilità traffico HTTP:**

- tcpdump cattura TUTTO il traffico in chiaro
- Credenziali facilmente estraibili
- Rischio elevato in reti non fidate (WiFi pubblico)

2. **Protezione traffico HTTPS:**

- Anche catturando i pacchetti, i dati restano illeggibili
- TLS/SSL offre protezione robusta
- Necessario certificato e chiave privata per decifrare

3. **Strumenti di analisi:**

- **tcpdump:** Cattura efficiente da riga di comando
- **Wireshark:** Analisi dettagliata e user-friendly
- Filtri essenziali per trovare informazioni rilevanti

4. **Sicurezza delle comunicazioni:**

- **Mai** trasmettere dati sensibili su HTTP
- Verificare sempre presenza HTTPS per login/pagamenti
- Consapevolezza dei rischi anche con HTTPS (phishing)

Raccomandazioni:

Per utenti:

- Usare sempre HTTPS per siti con credenziali
- Verificare certificato prima di inserire dati sensibili
- Evitare WiFi pubblici per operazioni critiche
- Usare VPN su reti non fidate

Per amministratori di rete:

- Implementare HTTPS su tutti i servizi web
- Configurare HSTS (HTTP Strict Transport Security)
- Monitorare traffico anomalo con IDS/IPS
- Educare utenti sui rischi di sicurezza

Per sviluppatori:

- Forzare redirect da HTTP a HTTPS
 - Usare certificati validi (non self-signed in produzione)
 - Implementare Certificate Pinning per app mobile
 - Seguire best practices OWASP
-

FILES GENERATI

File di cattura creati:

1. /home/analyst/httpdump.pcap (28 KB)
2. /home/analyst/httpsdump.pcap (89 KB)

Screenshot acquisiti: (se richiesti)

- Wireshark filtro HTTP
 - Credenziali in chiaro
 - Wireshark filtro HTTPS
 - Dati crittografati TLS
-

RIFERIMENTI

Comandi utilizzati:

Verifica interfacce

ip address

Cattura HTTP

sudo tcpdump -i eth0 -s 0 -w httpdump.pcap

Cattura HTTPS

sudo tcpdump -i eth0 -s 0 -w httpsdump.pcap

Analisi con Wireshark

wireshark httpdump.pcap

wireshark httpsdump.pcap

Filtri Wireshark:

- http → Filtra solo traffico HTTP
- tcp.port==443 → Filtra solo traffico HTTPS
- http.request.method == POST → Solo richieste POST
- tls.handshake → Solo handshake TLS

Siti testati:

- http://testphp.vulnweb.com/login.php (HTTP)
- https://www.netacad.com (HTTPS)

Report compilato da: Kali

Sistema: Kali Linux (VM)

Data completamento: 09/12/2025

Tempo totale esercizio: ~15 minuti

ESERCIZIO COMPLETATO CON SUCCESSO

Obiettivi raggiunti:

-  Cattura traffico HTTP con tcpdump
-  Analisi pacchetti HTTP con Wireshark
-  Identificazione credenziali in chiaro
-  Cattura traffico HTTPS con tcpdump
-  Analisi pacchetti HTTPS con Wireshark
-  Verifica crittografia TLS
-  Risposte a tutte le domande di riflessione

Competenze acquisite:

- Utilizzo tcpdump per network analysis

- Analisi protocolli HTTP/HTTPS con Wireshark
- Comprensione differenze sicurezza HTTP vs HTTPS
- Consapevolezza dei rischi di sicurezza web